

DDL, DML, Procedures and Variables

for Data Analysis

Denis Boldin



What we'll cover today

From changing tables to reusable database logic.

1

Structure vs data

DDL · DML
constraints

2

Safe changes

INSERT · UPDATE
DELETE · transactions

3

Reusable results

VIEW
MATERIALIZED VIEW

4

Stored logic

FUNCTION
PROCEDURE

5

Variables

local · session
client · parameters

6

Wrap-up

tool choice
Q & A

From reading data to changing it

Lecture 1 read the catalogue. Lecture 2 changes the database around it.

READ

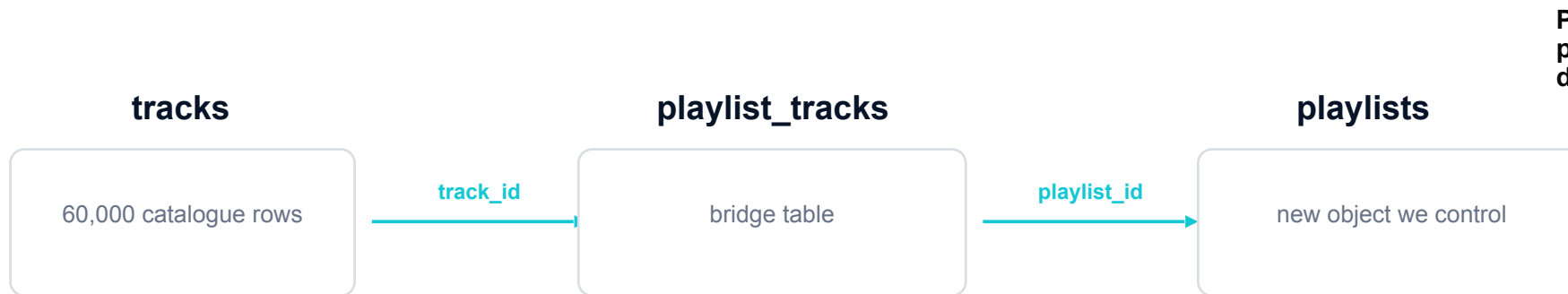
SELECT

Which rows?
Which metrics?

CHANGE

schema · rows · logic

We will build playlists instead of mutating the catalogue tables.



Playground reset: refresh the page to restore a clean database.

DDL vs DML

Start by asking what you are changing: the object or the rows inside it.

DDL

Data Definition Language

CREATE
ALTER
DROP
TRUNCATE

changes tables, indexes,
constraints and other objects

DML

Data Manipulation Language

INSERT
UPDATE
DELETE

changes the data inside
existing tables



Also in SQL

TCL

BEGIN · COMMIT · ROLLBACK

DCL

GRANT · REVOKE



CREATE

That works, but it says nothing about what is *valid*. In practice you also declare keys, length limits and constraints. The same table, written the way you would actually write it in PostgreSQL:

```
CREATE TABLE playlists (  
  id          SERIAL          PRIMARY KEY,  
  name        VARCHAR(120) NOT NULL,  
  owner       VARCHAR(80)  NOT NULL,  
  is_public   BOOLEAN       NOT NULL DEFAULT true,  
  created_at  TIMESTAMP     NOT NULL DEFAULT now(),  
  CONSTRAINT playlists_name_per_owner UNIQUE (name, owner)  
);  
  
INSERT INTO playlists (name, owner) VALUES  
  ('Late Night Drive', 'denis'),  
  ('Loud and Fast',    'denis'),  
  ('Quiet Mornings',  'anna');  
  
SELECT id, name, owner, is_public FROM playlists ORDER BY id;
```

RESULT

id	name	owner	is_public
1	Late Night Drive	denis	true
2	Loud and Fast	denis	true
3	Quiet Mornings	anna	true



CREATE

💡 DDL syntax differs considerably between DBMSs — ClickHouse, for example, additionally requires a table engine, a sort key and `ON CLUSTER` for a distributed rollout. Always consult the documentation of the system you are working with. Everything in this course is PostgreSQL.

Now the table that connects playlists to the catalogue — a bridge table, exactly like `track_artists` from Lecture 1:

```
CREATE TABLE playlist_tracks (  
    playlist_id INT          NOT NULL REFERENCES playlists(id) ON DELETE CASCADE,  
    track_id    TEXT         NOT NULL REFERENCES tracks(id),  
    position    SMALLINT    NOT NULL,  
    added_at    TIMESTAMP NOT NULL DEFAULT now(),  
    PRIMARY KEY (playlist_id, track_id)  
);  
  
SELECT count(*) AS rows_so_far FROM playlist_tracks;
```

RESULT

rows_so_far
0

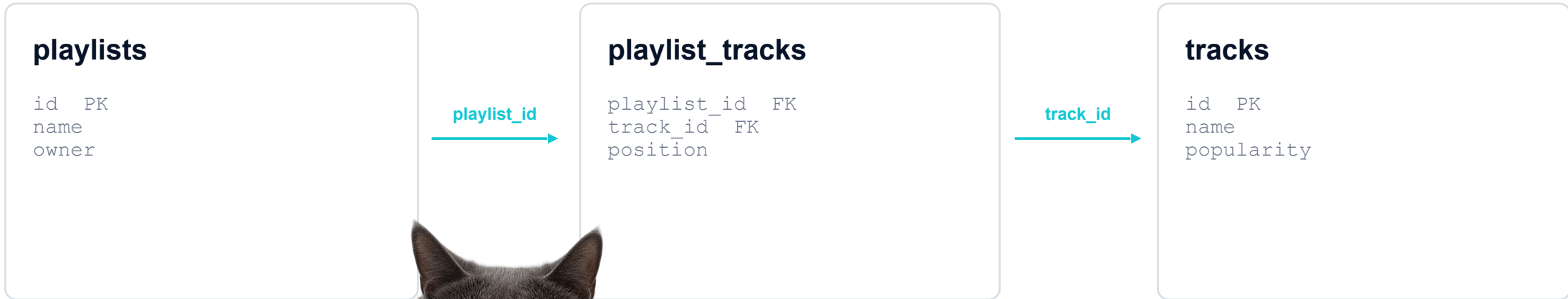


CREATE

- **PRIMARY KEY** uniquely identifies a row; there is exactly one per table, and it is **NOT NULL** by definition. Here it spans two columns — the same track cannot be added to the same playlist twice.
- **REFERENCES** (a foreign key) guarantees that the value exists in the referenced table — this is the "Consistency" of ACID being enforced for you.
- **ON DELETE CASCADE** says what to do when the parent goes: delete a playlist and its entries go with it. Without it, PostgreSQL would refuse the delete.
- **UNIQUE** forbids repeated values.
- **DEFAULT** supplies a value when the **INSERT** does not.

Foreign keys protect relationships

A bridge table should not be able to point at rows that do not exist.



```
playlist_id INT NOT NULL
REFERENCES playlists(id) ON DELETE CASCADE,
track_id TEXT NOT NULL REFERENCES tracks(id),
PRIMARY KEY (playlist_id, track_id)
```

Invalid track id → insert rejected

ON DELETE CASCADE removes bridge rows when the playlist is deleted.

ALTER TABLE: change the structure in place

Renames, new columns, defaults and type changes all use the same statement.

1

ADD COLUMN

```
ALTER TABLE playlists  
ADD COLUMN description text;
```

2

RENAME

```
ALTER TABLE playlists  
RENAME COLUMN description TO blurb;
```

3

SET DEFAULT

```
ALTER TABLE playlists  
ALTER COLUMN blurb SET DEFAULT '(no description)';
```

4

CHANGE TYPE

```
ALTER TABLE playlists  
ALTER COLUMN rating TYPE NUMERIC(3,1)  
USING rating::numeric;
```

Two details usually missed

DEFAULT applies only to new rows.

Existing rows keep NULL until you update them.

information_schema tells you what the database thinks the schema is.

INSERT can take rows from a query

The SELECT defines both the source rows and the values inserted.

Question

How do we build a running playlist from the catalogue?

```
INSERT INTO playlist_tracks
  (playlist_id, track_id, position)
SELECT 1,
       t.id,
       ROW_NUMBER() OVER (ORDER BY t.popularity DESC)
FROM tracks AS t
WHERE t.tempo BETWEEN 150 AND 170
      AND t.energy > 0.8
      AND t.popularity > 60
LIMIT 10;
```

Fields must match by position and type.

The literal 1 becomes playlist_id.

The query supplies track_id and position.

pos	track	pop.	bpm
1	Amor da Despedida	83	168
2	Último Beijo	83	157
3	Happy	81	160
4	Everlong	79	158
5	Giants	77	158



UPDATE

Changing data in all records of a column:

```
UPDATE playlists SET is_public = false;  
  
SELECT id, name, is_public FROM playlists ORDER BY id;
```

RESULT

id	name	is_public
1	Late Night Drive (renamed)	false
2	Loud and Fast	false
3	Quiet Mornings	false
4	Added Later	false



UPDATE

Changing data in filtered records:

```
UPDATE playlists
  SET is_public = true,
      blurb      = 'Curated by ' || owner
 WHERE owner = 'denis';

SELECT id, name, owner, is_public, blurb FROM playlists ORDER BY id;
```

RESULT

id	name	owner	is_public	blurb
1	Late Night Drive (renamed)	denis	true	Curated by denis
2	Loud and Fast	denis	true	Curated by denis
3	Quiet Mornings	anna	false	<i>NULL</i>
4	Added Later	denis	true	Curated by denis

Upsert and UPDATE FROM handle existing rows

Both patterns reuse data that is already in the database.

UPSERT

Insert — or update on conflict

```
INSERT INTO playlists (id, name, owner)
VALUES (1, 'Late Night Drive (renamed)', 'denis')
ON CONFLICT (id) DO UPDATE
SET name = EXCLUDED.name;
```

EXCLUDED = the row that would have been inserted.

UPDATE FROM

Update rows from a join

```
UPDATE playlists AS p
SET avg_popularity = stats.avg_pop
FROM (
    SELECT pt.playlist_id, round(avg(t.popularity),2) avg_pop
    FROM playlist_tracks pt JOIN tracks t ON t.id = pt.track_id
    GROUP BY pt.playlist_id
) AS stats
WHERE p.id = stats.playlist_id;
```

DELETE, TRUNCATE and safe changes

The fastest way to avoid a bad UPDATE or DELETE is to inspect the rows first.

DELETE

selective · row by row · supports WHERE and RETURNING

```
DELETE FROM playlists
WHERE owner = 'anna'
RETURNING id, name;
```

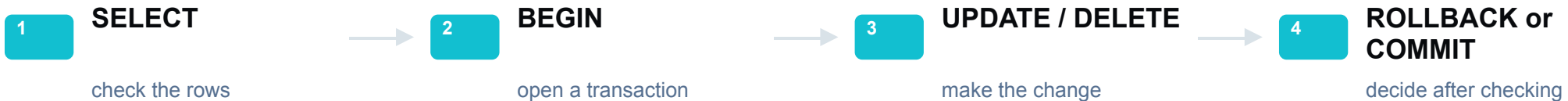
TRUNCATE

all rows · fast · structure stays

```
TRUNCATE TABLE playlists_copy;
```



A safe mutation workflow





DELETE

💡 `DELETE` should always have a condition. If you mean to delete everything, `TRUNCATE` is the right statement.

`DELETE ... RETURNING` gives back the rows it removed — useful when you want to log or archive them:

```
DELETE FROM playlist_tracks
WHERE playlist_id = 1
AND position > 8
RETURNING playlist_id, track_id, position;
```

RESULT

playlist_id	track_id	position
1	3rRin3LyLY92kpEbkCgwf4	9
1	74irxdVWstNIEQjsvArITq	10



A note on safety

Every DML statement in this section changes data. Two habits worth forming now:

1. Write the **SELECT** first, check what it returns, and only then convert it into a **DELETE** or an **UPDATE** with the same **WHERE**.
2. Wrap risky changes in a transaction, so a mistake can be undone:

```
BEGIN;
```

```
UPDATE playlists SET owner = 'oops';
```

```
SELECT id, name, owner FROM playlists ORDER BY id;    -- check the result
```

```
ROLLBACK;                                           -- or COMMIT if it looks right
```

```
SELECT id, name, owner FROM playlists ORDER BY id;    -- unchanged
```

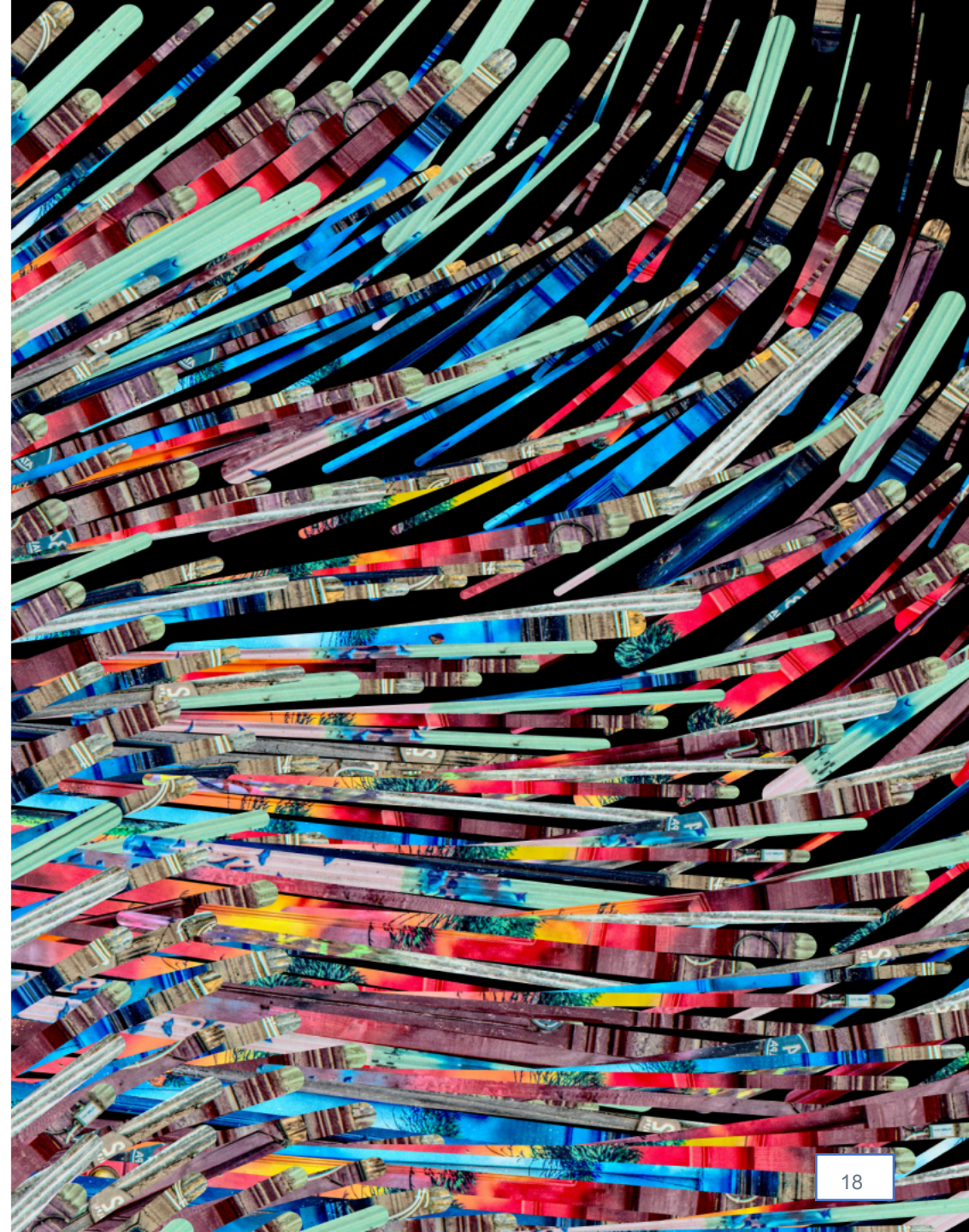
10-minute break

Back to views and stored logic after the break.



Views and materialized views

Name a query when you need it again.



A view is a reusable query

The data stays in the source tables. Reading the view runs its query again.

source tables

tracks

track_artists

artists

view definition

```
CREATE VIEW v_tracks_with_artists AS
SELECT t.id, t.name AS track,
       string_agg(a.name, ', ' ORDER BY
       ta.position) AS artists
FROM tracks t
JOIN track_artists ta ON ta.track_id = t.id
JOIN artists a ON a.id = ta.artist_id
GROUP BY t.id, t.name;
```

query the view

track	artists
Blinding Lights	The Weeknd
DÁKITI	Bad Bunny, Jhay Cortez
Anyone	Justin Bieber

VIEW stores the definition, not the rows.

Always current. No saved result. Recomputes on every read.

Unlike a CTE, a view persists in the database and can be reused by other queries.



Views

Replacing and dropping a view:

```
CREATE OR REPLACE VIEW v_tracks_with_artists AS  
SELECT ...;           -- the new definition  
  
DROP VIEW v_tracks_with_artists;
```

`CREATE OR REPLACE` can add columns to the end of a view but cannot rename or remove existing ones — for that you must `DROP` and re-create.



Materialized views

A **materialized view** looks like an ordinary one, but the result of the query is saved into a physical table stored on disk. That is the trade-off: reads become cheap, but the data is a snapshot and can go stale.

```
CREATE VIEW v_decade_counts AS
SELECT (EXTRACT(YEAR FROM release_date)::int / 10) * 10 AS decade,
       count(*) AS tracks
FROM tracks
GROUP BY 1;

CREATE MATERIALIZED VIEW mv_decade_counts AS
SELECT (EXTRACT(YEAR FROM release_date)::int / 10) * 10 AS decade,
       count(*) AS tracks
FROM tracks
GROUP BY 1;

SELECT 'view' AS source, tracks FROM v_decade_counts WHERE decade = 2020
UNION ALL
SELECT 'materialized',    tracks FROM mv_decade_counts WHERE decade = 2020;
```

RESULT

source	tracks
view	2068
materialized	2068



Materialized views

Identical, so far. Now add a track and ask both again — **without** refreshing:

```
INSERT INTO tracks (id, name, popularity, duration_ms, explicit,
                    release_date, release_date_precision)
VALUES ('demo0000000000000000000001', 'A Brand New Song', 0, 180000, false,
        DATE '2021-06-01', 'day');

SELECT 'view' AS source, tracks FROM v_decade_counts WHERE decade = 2020
UNION ALL
SELECT 'materialized',    tracks FROM mv_decade_counts WHERE decade = 2020;
```

RESULT

source	tracks
view	2069
materialized	2068

Materialized views store a snapshot

Reads are cheap. Freshness depends on when you refresh.

After inserting one 2021 track

VIEW runs query now

2069

MATERIALIZED reads stored rows

2068 **stale**



```
REFRESH MATERIALIZED VIEW mv_decade_counts;
```

View always current · query cost on every read

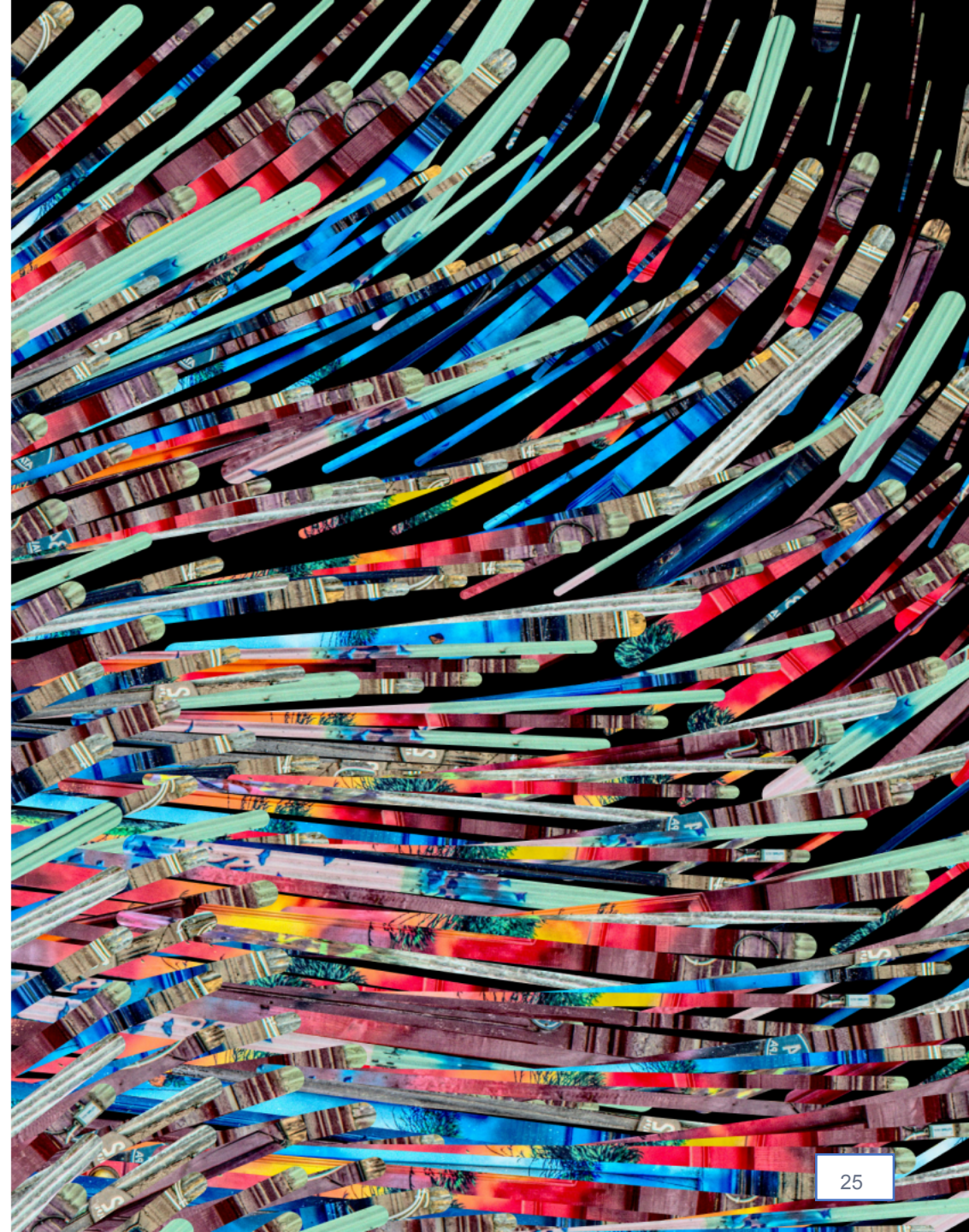
Materialized view stored rows · can be indexed · explicit refresh

Materialized views

	View	Materialized view	Table
Stores data	no	yes	yes
Cost of reading	the full query, every time	a read of stored rows	a read of stored rows
Freshness	always current	as of the last refresh	always current
Occupies disk	no (only the definition)	yes	yes
Can be indexed	no	yes	yes

Functions and procedures

Move reusable logic into the database.



Function or procedure?

The choice starts with what the caller expects back.

FUNCTION

returns a value or rows

```
SELECT decade_of(release_date);  
SELECT track_count('David Bowie');
```

usable inside another query

Need a value? **FUNCTION**

PROCEDURE

performs an action

```
CALL rebuild_decade_summary();
```

not usable inside a SELECT; may manage transactions

Need an action? **PROCEDURE**

A PL/pgSQL function has a clear structure

Variables live in DECLARE. Statements run inside BEGIN ... END.

```
CREATE OR REPLACE FUNCTION track_count(artist_name text)
RETURNS integer
LANGUAGE plpgsql
AS $$
DECLARE
    result integer;
BEGIN
    SELECT count(*) INTO result
    FROM artists a JOIN track_artists ta ON ta.artist_id = a.id
    WHERE a.name = artist_name;

    RETURN result;
END;
$$;
```

- 1 Signature** inputs + return type
- 2 DECLARE** local variables
- 3 BEGIN ... END** executable block
- 4 SELECT ... INTO** assign a query result

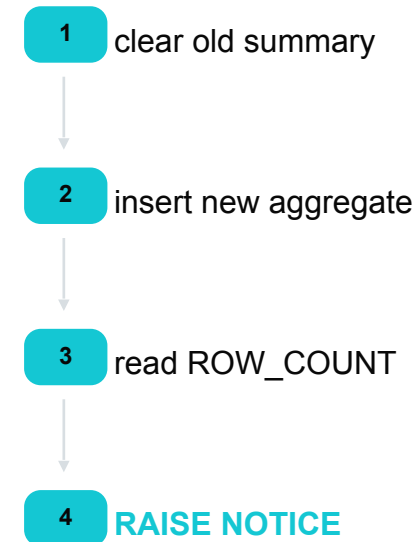
Dollar quoting (\$\$... \$\$) keeps the routine body readable.

A procedure performs an action

This one rebuilds a summary table and reports how many rows it wrote.

```
CREATE OR REPLACE PROCEDURE rebuild_decade_summary()  
LANGUAGE plpgsql AS $$  
DECLARE affected integer;  
BEGIN  
    DELETE FROM decade_summary;  
  
    INSERT INTO decade_summary (decade, tracks, avg_popularity)  
    SELECT decade_of(release_date), count(*), round(avg(popularity),2)  
    FROM tracks GROUP BY 1;  
  
    GET DIAGNOSTICS affected = ROW_COUNT;  
    RAISE NOTICE 'rebuilt with % rows', affected;  
END; $$
```

Action flow



NOTICE: rebuilt with 12 rows

Parameters, control flow and errors

Use procedural features when a single set-based statement is not enough.

PARAMETERS

IN input

OUT output

INOUT both

CONDITIONS

```
IF cnt = 0 THEN
  RETURN 'unknown';
ELSIF cnt > 100 THEN
  RETURN 'prolific';
ELSE
  RETURN 'established';
END IF;
```

ERROR HANDLING

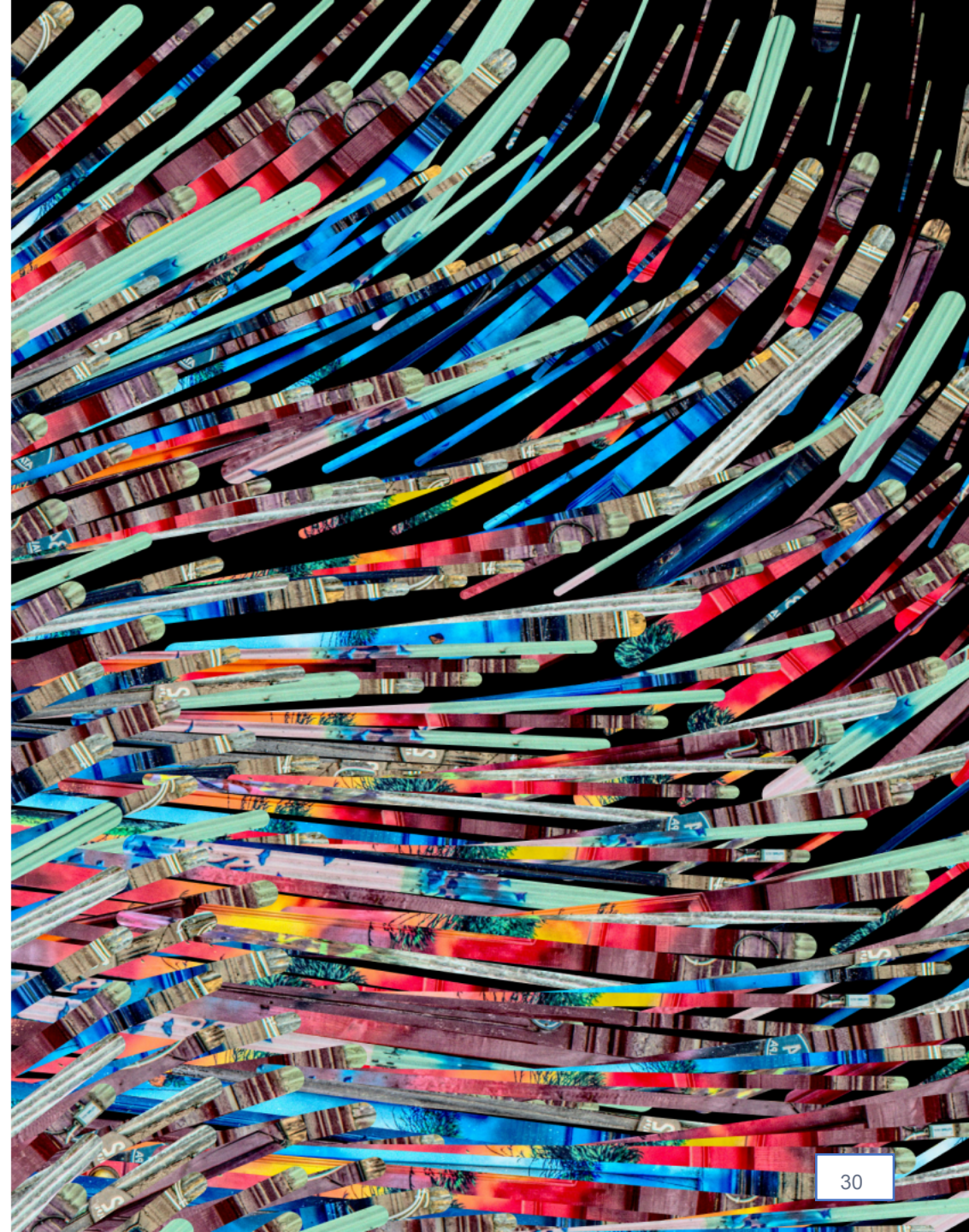
```
EXCEPTION
  WHEN unique_violation THEN
    RETURN 'already exists';
  WHEN others THEN
    RETURN SQLERRM;
```

Loops exist, but row-by-row work is usually slower than one set-based INSERT / UPDATE / SELECT.

Reach for FOR or WHILE only when the logic genuinely cannot be expressed as a set operation.

Variables

Variables inside PL/pgSQL
Session variables



Local variables live inside PL/pgSQL

A DO block lets you run procedural code once without creating a routine.

```
DO $$
DECLARE
  cutoff constant int := 80;
  hits integer;
  oldest tracks%ROWTYPE;
BEGIN
  SELECT count(*) INTO hits
  FROM tracks WHERE popularity >= cutoff;

  SELECT * INTO oldest
  FROM tracks ORDER BY release_date LIMIT 1;

  RAISE NOTICE 'hits: %', hits;
END;
$$;
```

Inside DECLARE

- ✓ := assigns a value
- ✓ SELECT ... INTO assigns a query result
- ✓ %TYPE follows one column type
- ✓ %ROWTYPE follows a whole table row
- ✓ record can hold a row of any shape

Scope: only while the routine or DO block is running.



An anonymous block: DO

A **DO** block takes no parameters and returns nothing — it acts. Its output goes to the notice channel, which the playground displays above the results.

Looping over a query, with a variable accumulating as it goes:

```
DO $$
DECLARE
    rec          record;
    running_total integer := 0;
BEGIN
    FOR rec IN
        SELECT decade_of(release_date) AS decade, count(*) AS tracks
        FROM tracks
        GROUP BY 1
        ORDER BY 1
    LOOP
        running_total := running_total + rec.tracks;
        RAISE NOTICE '% → % tracks (running total %)',
            rec.decade, rec.tracks, running_total;
    END LOOP;
END;
$$;
```

Session, psql and application parameters

Choose the mechanism by where the value needs to live.

SESSION

```
SET my.decade = '1970';  
current_setting('my.decade')
```

one connection

SET LOCAL = one transaction

PSQL

```
\set decade 1970  
... WHERE decade = :decade
```

client script

not available in the browser playground

APPLICATION

```
... WHERE name = $1
```

query call

safe parameter, never parsed as SQL

Security rule

Never build SQL by concatenating user input. Pass values as \$1, \$2, ...

How to choose the right tool

Start with what must change or what must be reused.

1 Change database structure? **DDL**

CREATE · ALTER · DROP · TRUNCATE

2 Change rows? **DML**

INSERT · UPDATE · DELETE

3 Reuse a query later? **VIEW**

named query, always current

4 Need stored results? **MATERIALIZED VIEW**

fast reads, explicit refresh

5 Need a value in SQL? **FUNCTION**

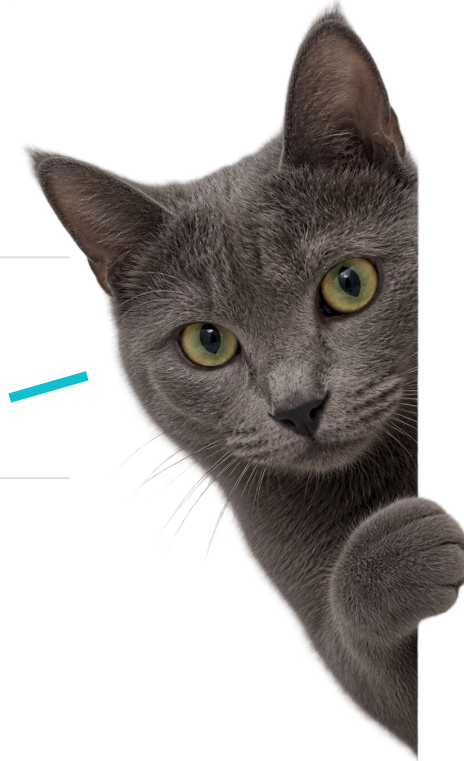
returns a value or rows

6 Need an action? **PROCEDURE**

CALL it

7 Need temporary state? **VARIABLE**

local · session · client



Good database changes make invalid states hard to create and risky changes easy to undo.

Q & A

Denis Boldin · PostgreSQL

